

学号：22920202200773

姓名：孟媛

实验二 牛顿迭代法求方程的根

一、实验目的

使用牛顿迭代法求方程的根。

二、数值计算原理

设 $f(x)$ 连续可导， x^* 是 $f(x)$ 的一个根， x_k 是其一个近似解，将 $f(x)$ 在 x_k 处做一阶泰勒展开，则有：
$$f(x) \approx f(x_k) + f'(x_k)(x - x_k)$$

于是有：
$$0 = f(x) \approx f(x_k) + f'(x_k)(x - x_k)$$

当 $f'(x) \neq 0$ 时， $x^* \approx x_k - \frac{f(x_k)}{f'(x_k)}$ ，右式可以看成是 x^* 的一个新近似值 x_{k+1} ，则获得迭

代公式 $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ 。

三、源程序介绍

函数接受三个参数：求根区间下限、求根区间上限、精度，输出求根过程，返回最终求得的足够精度的根。

求解过程如下：

- (1) 新值和旧值的差小于设定的精度，则结束操作，输出结果
- (2) 利用公式 $x_i = x_{i-1} - \frac{f(x_{i-1})}{f'(x_{i-1})}$ ($i=1,2,\dots$) 更新 x ， $\text{cnt}++$
- (3) 转到 (1)

```

y > ...
#牛顿迭代法求方程的根
import numpy
import math
from scipy import optimize

cnt = 0 #计数迭代次数

def func(x): #原函数
    return math.pow(x,3) - x - 1

def func_(x): #导函数
    return 3 * math.pow(x, 2) - 1

def check(l, r, st): #参数依次为左区间,右区间,精度
    global cnt
    x = r
    while(numpy.fabs((func(x) / func_(x))) > st):
        cnt += 1
        x -= func(x) / func_(x)
    return x

if __name__ == '__main__':
    l = float(input("请输入左区间, 以回车结束:"))
    r = float(input("请输入右区间, 以回车结束:"))
    st = float(input("请输入精度:"))
    print("自编函数求得方程的解为:", check(l, r, st), ", 迭代进行了:", cnt, "次")
    print("自带函数求得方程的解为:", optimize.newton(func, x0 = l, x1 = r, tol = st))

```

四、程序执行情况

4.1 书上给的例子:

```

def func(x): #原函数
    return math.pow(x,3) - x - 1

def func_(x): #导函数
    return 3 * math.pow(x, 2) - 1

```

```

请输入左区间, 以回车结束:1
请输入右区间, 以回车结束:10
请输入精度:0.0000001
自编函数求得方程的解为: 1.3247179573196575 , 迭代进行了: 9 次
自带函数求得方程的解为: 1.3247179572447472

```

4.2 书上未给的例子

```
def func(x): #原函数
    return numpy.sin(x) - x * x / 4

def func_(x): #导函数
    return numpy.cos(x) - x / 2
```

请输入左区间，以回车结束:1

请输入右区间，以回车结束:10

请输入精度:0.00000001

自编函数求得方程的解为: 1.9337537628275308 , 迭代进行了: 7 次

自带函数求得方程的解为: 1.9337537628270212

五、结果分析

相较二分法，牛顿迭代法可以用较少的迭代次数获得精度较高的解。

但牛顿法迭代法不一定能保证找到解。初始点的选取不当会使迭代过程是发散的。此外，当迭代过程某点的导数为零，迭代将无法继续进行。

工具函数较为方便，自编函数需要手动求导，正确性可能受求导错误影响。

六、实验体会

牛顿迭代法思想也比较简单，但是可能求不出解。

本次实验，我对牛顿迭代法的理解更加深入，并与二分法进行对比，体会到了牛顿迭代法的高效。总体来说还是一种比较好的方法。